PRACTICAL MODULE

Introduction to APIs with Plain PHP

Concepts, types of APIs, building a simple REST API, and consuming public APIs

AI Disclaimer
This course is delivered by Agus Budi Raharjo. Claude AI assisted in developing the content and structure of this course. Agus Budi Raharjo is solely responsible for all of the course content.

Learning Objectives

By the end of this session, students can:

1. Explain what an API is and the roles of client and server.
2. Distinguish the main types of APIs (by access level and by architecture).
3. Build simple REST API endpoints (CRUD) in plain PHP that return JSON.
4. Test an API using the browser and a testing tool (cURL/Postman).
5. Consume a public API with PHP cURL and display the data on a web page.

Preparation Before Class

- Start **Apache** and **MySQL** in the XAMPP Control Panel.
- Make sure the cURL extension is enabled: open `php.ini` (XAMPP → Config → PHP) and check that the line `extension=curl` has no leading semicolon. Restart Apache if you change it.
- Create the project folder `C:\xampp\htdocs\api-demo`.
- Install Postman or the Thunder Client extension for VS Code (optional; cURL also works).

1. What Is an API?

An **API (Application Programming Interface)** lets one program communicate with another without knowing how it works internally. On the web, an API is usually a **URL (endpoint)** that accepts a request and returns data, most often as JSON.

> **Analogy: A Restaurant**
> **Customer** = client (browser, mobile app, PHP script).
> **Waiter** = API (takes the order, delivers the result).
> **Kitchen** = server and database (where data is processed).
> The customer never enters the kitchen; they order through the waiter using a clear menu (the API documentation).

1.1 How It Works

```
1. Client  ->  Request  : GET /mahasiswa.php?id=1
2. Server (PHP)  ->  Queries the MySQL database
3. Server  ->  Client   : Response 200 OK + JSON data
```

1.2 Key Components

An **endpoint** is the address of a resource, for example `http://localhost/api-demo/mahasiswa.php`. The **HTTP method** states the requested action:

Method	Purpose	CRUD equivalent	Example
GET	Retrieve data	Read	Get the list of students
POST	Add data	Create	Add a new student
PUT	Modify data	Update	Change the name of student 3
DELETE	Remove data	Delete	Delete student 3

Status codes tell the client how the request went:

Code	Meaning	When it is used
200	OK	The request succeeded
201	Created	New data was created (POST)
400	Bad Request	Input is wrong or incomplete
404	Not Found	Data or endpoint was not found
405	Method Not Allowed	The endpoint does not support that method
500	Server Error	Something failed on the server

JSON (JavaScript Object Notation) is a lightweight, human-readable data exchange format. In PHP:

```php
<?php
$data = ['nim' => '2024001', 'nama' => 'Budi'];
echo json_encode($data);             // PHP array  -> JSON string
$obj = json_decode('{"a":1}', true); // JSON string -> PHP array
```

Quick demo

Open `https://jsonplaceholder.typicode.com/posts/1` in the browser. It shows that a public API is just a URL that returns JSON.

2. Types of APIs

2.1 By Access Level

Type	Description	Examples
Public / Open API	Open to everyone, sometimes requires an API key	Open-Meteo (weather), JSONPlaceholder, REST Countries
Partner API	Shared only with specific partners	Payment API for registered merchants
Private / Internal API	Used inside an organization	API between your own web and mobile apps
Composite API	Combines several API calls into one	One request returning user + order data

2.2 By Architecture / Protocol

Type	Characteristics	Data format
REST	URL + HTTP method based, stateless, most popular on the web	JSON (sometimes XML)
SOAP	Strict protocol with a WSDL contract, common in legacy/banking systems	XML
GraphQL	Single endpoint; the client chooses which fields to request	JSON
WebSocket	Two-way real-time connection (chat, notifications)	Any
gRPC	High performance between services (microservices)	Protocol Buffers

Focus today

We use **REST APIs with JSON**, because they are the most common and the easiest to practice with plain PHP.

2.3 REST Principles Used in the Practice

- Each resource has its own URL (for example `/mahasiswa.php?id=3`).
- The action is determined by the HTTP method, not by the file or URL name.
- The server keeps no client state (stateless): every request carries enough information.
- Responses are always JSON with the correct status code.

3. Practice: Building an API with Plain PHP

We build a student data API with full CRUD operations.

Step 1 – Prepare the Database

Open phpMyAdmin (`http://localhost/phpmyadmin`) → SQL tab, then run:

SQL

```sql
CREATE DATABASE kuliah_api CHARACTER SET utf8mb4;
USE kuliah_api;

CREATE TABLE mahasiswa (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nim VARCHAR(15) NOT NULL UNIQUE,
  nama VARCHAR(100) NOT NULL,
  jurusan VARCHAR(50)
);

INSERT INTO mahasiswa (nim, nama, jurusan) VALUES
('2024001', 'Budi Santoso', 'Informatika'),
('2024002', 'Sari Dewi', 'Sistem Informasi');
```

Step 2 – Database Connection

api-demo/koneksi.php

```php
<?php
$pdo = new PDO(
  "mysql:host=localhost;dbname=kuliah_api;charset=utf8mb4",
  "root", "",
  [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
  ]
);
```

Step 3 – The API Endpoint

api-demo/mahasiswa.php

```php
<?php
header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *');   // allow access from other origins
require 'koneksi.php';

function kirim($data, $kode = 200) {        // send JSON response
  http_response_code($kode);
  echo json_encode($data, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
  exit;
}

$method = $_SERVER['REQUEST_METHOD'];
$id     = isset($_GET['id']) ? (int) $_GET['id'] : null;
$input  = json_decode(file_get_contents('php://input'), true) ?? [];

try {
  switch ($method) {

    case 'GET':
      if ($id) {
        $st = $pdo->prepare("SELECT * FROM mahasiswa WHERE id = ?");
        $st->execute([$id]);
        $row = $st->fetch();
        if (!$row) kirim(['message' => 'Data not found'], 404);
        kirim($row);
      }
```

```
      kirim($pdo->query("SELECT * FROM mahasiswa")->fetchAll());

    case 'POST':
      if (empty($input['nim']) || empty($input['nama'])) {
        kirim(['message' => 'nim and nama are required'], 400);
      }
      $st = $pdo->prepare(
        "INSERT INTO mahasiswa (nim, nama, jurusan) VALUES (?, ?, ?)");
      $st->execute([$input['nim'], $input['nama'], $input['jurusan'] ?? null]);
      kirim(['message' => 'Data created', 'id' => $pdo->lastInsertId()], 201);

    case 'PUT':
      if (!$id) kirim(['message' => 'id parameter is required'], 400);
      $st = $pdo->prepare(
        "UPDATE mahasiswa SET nama = ?, jurusan = ? WHERE id = ?");
      $st->execute([$input['nama'] ?? '', $input['jurusan'] ?? null, $id]);
      kirim(['message' => 'Data updated']);

    case 'DELETE':
      if (!$id) kirim(['message' => 'id parameter is required'], 400);
      $pdo->prepare("DELETE FROM mahasiswa WHERE id = ?")->execute([$id]);
      kirim(['message' => 'Data deleted']);

    default:
      kirim(['message' => 'Method not allowed'], 405);
  }
} catch (PDOException $e) {
  kirim(['message' => 'Server error'], 500);
}
```

Step 4 – Test the API

GET in the browser:

- `http://localhost/api-demo/mahasiswa.php` → list of all students
- `http://localhost/api-demo/mahasiswa.php?id=1` → a single student

POST, PUT, and DELETE cannot be tried from the address bar. Use Postman/Thunder Client (Body → raw → JSON) or cURL in the Windows Command Prompt:

Add data (POST)

```
curl.exe -X POST "http://localhost/api-demo/mahasiswa.php" -H "Content-Type: application/json" -d
"{\"nim\":\"2024003\",\"nama\":\"Andi\",\"jurusan\":\"Informatika\"}"
```

Update data (PUT)

```
curl.exe -X PUT "http://localhost/api-demo/mahasiswa.php?id=3" -H "Content-Type: application/json"
-d "{\"nama\":\"Andi Pratama\",\"jurusan\":\"Informatika\"}"
```

Delete data (DELETE)

```
curl.exe -X DELETE "http://localhost/api-demo/mahasiswa.php?id=3"
```

Discussion points

Why use **prepared statements** (`?`)? → they prevent SQL injection.

Why different status codes (201, 400, 404)? → clients can react automatically without parsing the message.

What is the risk of `Access-Control-Allow-Origin: *` in production? → anyone can call the API from a browser; restrict domains and add authentication.

4. Practice: Consuming a Public API

Consuming an API means our application acts as a client that calls someone else's API, then processes and displays the data. In PHP the usual tool is **cURL** (or `file_get_contents` for simple cases).

4.1 Quick Way: file_get_contents

api-demo/consume1.php

```php
<?php
$url  = "https://jsonplaceholder.typicode.com/posts?_limit=5";
$json = file_get_contents($url);
$data = json_decode($json, true);

foreach ($data as $post) {
  echo "<h3>" . htmlspecialchars($post['title']) . "</h3>";
  echo "<p>" . htmlspecialchars($post['body']) . "</p>";
}
```

4.2 Standard Way: cURL (with error handling)

This example uses **Open-Meteo**, a free weather API that needs no API key. Jakarta coordinates: latitude -6.2, longitude 106.8.

api-demo/cuaca.php

```php
<?php
function ambilApi($url) {
  $ch = curl_init($url);
  curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_TIMEOUT        => 10,
    CURLOPT_HTTPHEADER     => ['Accept: application/json'],
  ]);
  $respon = curl_exec($ch);
  $kode   = curl_getinfo($ch, CURLINFO_HTTP_CODE);
  $error  = curl_error($ch);
  curl_close($ch);

  if ($respon === false) return ['error' => $error];
  if ($kode !== 200)     return ['error' => "HTTP $kode"];
  return json_decode($respon, true);
}

$url = "https://api.open-meteo.com/v1/forecast"
     . "?latitude=-6.2&longitude=106.8"
     . "&current=temperature_2m,wind_speed_10m";
$cuaca = ambilApi($url);
?>
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>Jakarta Weather</title></head>
<body>
  <h2>Current Weather in Jakarta</h2>
  <?php if (isset($cuaca['error'])): ?>
    <p>Failed to fetch data: <?= htmlspecialchars($cuaca['error']) ?></p>
  <?php else: ?>
    <p>Temperature: <b><?= $cuaca['current']['temperature_2m'] ?> °C</b></p>
    <p>Wind speed: <b><?= $cuaca['current']['wind_speed_10m'] ?> km/h</b></p>
  <?php endif; ?>
</body>
</html>
```

4.3 Consuming Your Own API

Our own API is also just a URL, so calling it works the same way. Show the student data in a table:

api-demo/tampil.php

```php
<?php
$data = json_decode(
  file_get_contents("http://localhost/api-demo/mahasiswa.php"), true);
?>
<table border="1" cellpadding="6">
  <tr><th>NIM</th><th>Name</th><th>Major</th></tr>
  <?php foreach ($data as $m): ?>
  <tr>
    <td><?= htmlspecialchars($m['nim']) ?></td>
    <td><?= htmlspecialchars($m['nama']) ?></td>
    <td><?= htmlspecialchars($m['jurusan']) ?></td>
  </tr>
  <?php endforeach; ?>
</table>
```

Important tips

Always check for errors and the status code before using the data.

Always wrap output in `htmlspecialchars()` to prevent XSS.

Public APIs usually have rate limits and documentation; read them before use.

If an API needs a key, keep it in a config file that is not pushed to a public repository.

5. Exercises, Quiz, and Wrap-Up

5.1 Individual Exercises

Work in class (any remaining time becomes homework):

1. Add one student with POST, change it with PUT, then delete it with DELETE. Note the status code of each response.
2. Modify `mahasiswa.php` so that GET supports searching: `?nama=budi` (use `LIKE`).
3. Build a `negara.php` page that consumes `https://restcountries.com/v3.1/name/indonesia` and shows the official name, capital, and population.
4. Challenge: build an HTML form that adds a student by sending data to the API with cURL (POST method, JSON body).

5.2 Short Quiz

1. What does API stand for, and what is its main purpose?
2. Name the 4 HTTP methods and their CRUD equivalents.
3. What is the difference between status codes 201, 400, and 404?
4. How do REST and SOAP differ in data format?
5. Which PHP functions convert an array to JSON and back?
6. Why should a third-party API response be checked before it is displayed?

5.3 Summary

- An API is a communication bridge between applications; on the web it is usually a URL that returns JSON.
- REST uses HTTP methods and status codes as a standard language.
- Building an API in plain PHP = read the method/input, process against the database, then `json_encode` with the right status code.
- Consuming an API = send a request (cURL), check the result, `json_decode`, display it safely.

5.4 Assignment Rubric

Aspect	Weight	Criteria
Working CRUD API (GET/POST/PUT/DELETE)	35%	All methods work, status codes are appropriate
Public API consumption	25%	Data displays correctly, errors are handled
Basic security	20%	Prepared statements, htmlspecialchars
Code quality & endpoint documentation	20%	Well structured, endpoint list in a README

6. Case Study Project: Kopipiro Coffee Shop

6.1 The Case

The owner of **Kopipiro** needs a coffee ordering web application. The admin (owner) adds products, uploads images and sets prices. A buyer who registers and logs in gets a **10% discount**; a visitor who orders without logging in pays the normal price. Besides coffee, Kopipiro sells toast, non-caffeine drinks and more.

Requirement	How we build it
1. Explore products (coffee / non-coffee) with description, rating, price	`GET api/products.php` with category filter; ratings come from a `reviews` table
2. Order multiple products	Cart in the browser, sent as a list of items to `POST api/orders.php`
3. Checkout shows prices including discount	The API returns `final_price` for the current user; the server recalculates the real total
4. Users: Owner, Customer, Cashier	`role` column + `need_role()` check inside every protected endpoint
5. Cashier sees only their own orders	Each order gets a `cashier_id`; the orders API filters by it
6. Owner sees all orders, sales charts, Excel download	`api/sales.php` + Chart.js, and `api/export.php` with PhpSpreadsheet

6.2 API Design

Endpoint	Method	Who	Purpose
api/auth.php?action=register	POST	Anyone	Create a customer account
api/auth.php?action=login	POST	Anyone	Start a session
api/auth.php?action=me / logout	GET / POST	Anyone	Current user / end session
api/products.php	GET	Anyone	List products (discounted price if customer)
api/products.php	POST	Owner	Add product + image upload
api/products.php?id=3	PUT	Owner	Change price
api/orders.php	POST	Guest or customer	Create an order
api/orders.php	GET	Owner / cashier / customer	List orders (each role sees its own slice)
api/orders.php?id=5	PUT	Owner / cashier	Update status (pending, paid, done)
api/sales.php	GET	Owner	Chart data

Endpoint	Method	Who	Purpose
api/export.php	GET	Owner	Download Excel
api/pay.php	POST	Anyone	Create a Midtrans sandbox payment
api/midtrans_notify.php	POST	Midtrans only	Payment notification (webhook)

Key design rule

Never trust the browser. The browser shows prices for convenience, but the server always reads prices from the database and applies the discount itself. Likewise, roles are enforced in the API, not just by hiding buttons.

6.3 Project Structure

```
C:\xampp\htdocs\kopipiro\
  config.php            database connection + settings
  helpers.php           JSON output, roles, discount rule
  shop.html             storefront (catalog, cart, checkout, login)
  dashboard.html        cashier and owner dashboard
  api\
    auth.php  products.php  orders.php  sales.php
    export.php  pay.php  midtrans_notify.php
  tools\
    seed_base.php  seed_from_api.php
  uploads\              product images uploaded by the owner
```

Create the folders first. The code in this section is complete: each file below is used exactly as shown.

Step 1 – Database

In phpMyAdmin, open the SQL tab and run the script below. Note that `order_items` stores a snapshot of the product name and price, so old orders stay correct even if the owner later changes a price.

SQL

```
CREATE DATABASE kopipiro CHARACTER SET utf8mb4;
USE kopipiro;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('owner','cashier','customer') NOT NULL DEFAULT 'customer'
);

CREATE TABLE categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  price INT NOT NULL,                 -- normal price in Rupiah
```

```
  image VARCHAR(255),                -- uploaded file name OR full URL
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);

CREATE TABLE reviews (
  id INT AUTO_INCREMENT PRIMARY KEY,
  product_id INT NOT NULL,
  user_id INT NOT NULL,
  rating TINYINT NOT NULL,           -- 1..5
  comment VARCHAR(255),
  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_code VARCHAR(30) NOT NULL UNIQUE,
  user_id INT NULL,                  -- NULL = guest
  customer_name VARCHAR(100) NOT NULL,
  cashier_id INT NULL,
  subtotal INT NOT NULL,
  discount INT NOT NULL DEFAULT 0,
  total INT NOT NULL,
  status ENUM('pending','paid','done') NOT NULL DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (cashier_id) REFERENCES users(id)
);

CREATE TABLE order_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  product_name VARCHAR(100) NOT NULL, -- snapshot at order time
  unit_price INT NOT NULL,           -- normal price at order time
  qty INT NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(id)
);
```

Step 2 – Config and Helpers

config.php

```
<?php
session_start();

$pdo = new PDO(
  "mysql:host=localhost;dbname=kopipiro;charset=utf8mb4",
  "root", "",
  [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
    PDO::MYSQL_ATTR_FOUND_ROWS   => true,
  ]
);

const DISCOUNT_RATE = 0.10;                    // 10% for logged-in customers
const MIDTRANS_SERVER_KEY = 'PASTE-YOUR-SANDBOX-SERVER-KEY';
```

`helpers.php` holds the small functions every endpoint shares. The discount rule lives here and nowhere else.

helpers.php

```
<?php
require_once __DIR__ . '/config.php';
header('Content-Type: application/json; charset=utf-8');
```

```
function out($data, int $code = 200): void {
  http_response_code($code);
  echo json_encode($data, JSON_UNESCAPED_UNICODE);
  exit;
}

function body(): array {                          // JSON request body
  static $b = null;
  if ($b === null) {
    $b = json_decode(file_get_contents('php://input'), true) ?? [];
  }
  return $b;
}

function me(): ?array {
  return $_SESSION['user'] ?? null;
}

function need_role(array $roles): array {
  $u = me();
  if (!$u) out(['message' => 'Please log in'], 401);
  if (!in_array($u['role'], $roles, true)) out(['message' => 'Forbidden'], 403);
  return $u;
}

// The discount rule lives ONLY here, on the server.
function final_price(int $price, ?array $user): int {
  $member = $user && $user['role'] === 'customer';
  return $member ? (int) round($price * (1 - DISCOUNT_RATE)) : $price;
}

function image_url(?string $img): ?string {
  if (!$img) return null;
  return preg_match('#^https?://#', $img) ? $img : 'uploads/' . $img;
}
```

Step 3 – Seed Data (and Your First Public API)

We need demo users and products. The coffee menu is imported from a **public API** (Sample APIs Coffee), which is real practice in consuming an API. That API has no prices, so we invent them. Other products (non-coffee, toast) are added by hand.

tools/seed_base.php

```
<?php
// Run once:  C:\xampp\php\php.exe tools\seed_base.php
require __DIR__ . '/../config.php';

foreach (['Coffee', 'Non-Coffee', 'Food'] as $c) {
  $pdo->prepare("INSERT IGNORE INTO categories (name) VALUES (?)")->execute([$c]);
}

$users = [
  ['Kopipiro Owner', 'owner@kopipiro.test',    'owner123',    'owner'],
  ['Cashier Ani',    'cashier1@kopipiro.test', 'cashier123',  'cashier'],
  ['Cashier Budi',   'cashier2@kopipiro.test', 'cashier123',  'cashier'],
  ['Demo Customer',  'customer@kopipiro.test', 'customer123', 'customer'],
];
$ins = $pdo->prepare(
  "INSERT IGNORE INTO users (name, email, password_hash, role) VALUES (?, ?, ?, ?)");
foreach ($users as [$name, $email, $pass, $role]) {
  $ins->execute([$name, $email, password_hash($pass, PASSWORD_DEFAULT), $role]);
}

// Non-coffee products, added by hand (the public coffee API has none).
$find = $pdo->prepare("SELECT id FROM categories WHERE name = ?");
$cat = function ($name) use ($find) {
  $find->execute([$name]);
  return $find->fetchColumn();
```

```
};
$items = [
  ['Non-Coffee', 'Chocolate Milk',   'Cold chocolate with fresh milk',        22000],
  ['Non-Coffee', 'Matcha Latte',     'Japanese green tea with steamed milk',  26000],
  ['Non-Coffee', 'Lemon Tea',        'Black tea with fresh lemon',            15000],
  ['Food',       'Butter Toast',     'Thick toast with butter and sugar',     18000],
  ['Food',       'Chocolate Toast',  'Thick toast with chocolate and cheese', 20000],
];
$ins = $pdo->prepare(
  "INSERT INTO products (category_id, name, description, price) VALUES (?, ?, ?, ?)");
foreach ($items as [$c, $name, $desc, $price]) {
  $ins->execute([$cat($c), $name, $desc, $price]);
}
echo "Base data ready.\n";
```

tools/seed_from_api.php

```
<?php
// Run once, AFTER seed_base.php:  C:\xampp\php\php.exe tools\seed_from_api.php
// Consumes a public API (no key needed) and copies the data into OUR database.
require __DIR__ . '/../config.php';

$ch = curl_init('https://api.sampleapis.com/coffee/hot');
curl_setopt_array($ch, [CURLOPT_RETURNTRANSFER => true, CURLOPT_TIMEOUT => 20]);
$resp = curl_exec($ch);
$code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

$list = json_decode((string) $resp, true);
if ($code !== 200 || !is_array($list)) {
  exit("Public API failed (HTTP $code). Try again later.\n");
}

$coffee = $pdo->query("SELECT id FROM categories WHERE name = 'Coffee'")->fetchColumn();
$exists = $pdo->prepare("SELECT COUNT(*) FROM products WHERE name = ?");
$ins = $pdo->prepare(
  "INSERT INTO products (category_id, name, description, price, image) VALUES (?, ?, ?, ?, ?)");

foreach ($list as $c) {
  $exists->execute([$c['title']]);
  if ($exists->fetchColumn()) continue;
  $desc  = $c['description'] . ' Ingredients: ' . implode(', ', $c['ingredients'] ?? []);
  $price = 20000 + ($c['id'] % 6) * 2500;        // the API has no prices, so we invent them
  $ins->execute([$coffee, $c['title'], $desc, $price, $c['image'] ?? null]);
}

// Demo ratings so the catalog shows stars (3 random 3-5 ratings per product).
$customer = $pdo->query("SELECT id FROM users WHERE role = 'customer' LIMIT 1")->fetchColumn();
$rate = $pdo->prepare("INSERT INTO reviews (product_id, user_id, rating) VALUES (?, ?, ?)");
foreach ($pdo->query("SELECT id FROM products WHERE id NOT IN (SELECT product_id FROM reviews)")
         ->fetchAll(PDO::FETCH_COLUMN) as $pid) {
  for ($i = 0; $i < 3; $i++) $rate->execute([$pid, $customer, random_int(3, 5)]);
}
echo "Imported " . count($list) . " coffees from the public API.\n";
```

Run them from Command Prompt:

```
cd C:\xampp\htdocs\kopipiro
C:\xampp\php\php.exe tools\seed_base.php
C:\xampp\php\php.exe tools\seed_from_api.php
```

Account	Email	Password	Role
Owner	owner@kopipiro.test	owner123	owner
Cashier Ani	cashier1@kopipiro.test	cashier123	cashier
Cashier Budi	cashier2@kopipiro.test	cashier123	cashier

Account	Email	Password	Role
Demo Customer	customer@kopipiro.test	customer123	customer

> **If the public API is down**
>
> Free public APIs can be slow or unavailable. The script stops with a clear message; just run it again later. This is also a good moment to discuss why we **copy** the data into our own database instead of calling the public API on every page view.

Step 4 – Authentication API

Passwords are stored with `password_hash()` and checked with `password_verify()`. Self-registration always creates a **customer**; owner and cashier accounts are created only by the seed script (or by an admin).

api/auth.php

```php
<?php
require '../helpers.php';

$action = $_GET['action'] ?? '';
$in = body();

switch ($action) {
  case 'register':
    $name  = trim($in['name'] ?? '');
    $email = strtolower(trim($in['email'] ?? ''));
    $pass  = $in['password'] ?? '';
    if ($name === '' || !filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($pass) < 6) {
      out(['message' => 'Name, valid email and password (min 6 chars) are required'], 400);
    }
    try {
      $st = $pdo->prepare(
        "INSERT INTO users (name, email, password_hash, role) VALUES (?, ?, ?, 'customer')");
      $st->execute([$name, $email, password_hash($pass, PASSWORD_DEFAULT)]);
    } catch (PDOException $e) {
      if ($e->errorInfo[1] == 1062) out(['message' => 'Email already registered'], 409);
      throw $e;
    }
    out(['message' => 'Registered. You can log in now.'], 201);

  case 'login':
    $st = $pdo->prepare("SELECT * FROM users WHERE email = ?");
    $st->execute([strtolower(trim($in['email'] ?? ''))]);
    $u = $st->fetch();
    if (!$u || !password_verify($in['password'] ?? '', $u['password_hash'])) {
      out(['message' => 'Wrong email or password'], 401);
    }
    session_regenerate_id(true);
    $_SESSION['user'] = ['id' => (int) $u['id'], 'name' => $u['name'], 'role' => $u['role']];
    out($_SESSION['user']);

  case 'logout':
    session_destroy();
    out(['message' => 'Logged out']);

  case 'me':
    out(me() ?? ['role' => 'guest']);

  default:
    out(['message' => 'Unknown action'], 404);
}
```

Step 5 – Products API (Discount and Image Upload)

GET returns both `price` (normal) and `final_price` (what this visitor pays). POST lets only the owner add a product, with strict image validation: size limit, real MIME type check, and a random file name.

api/products.php

```php
<?php
require '../helpers.php';
$user = me();

switch ($_SERVER['REQUEST_METHOD']) {

  case 'GET':
    $sql = "SELECT p.id, p.name, p.description, p.price, p.image, c.name AS category,
                   ROUND(AVG(r.rating), 1) AS rating, COUNT(r.id) AS reviews
            FROM products p
            JOIN categories c ON c.id = p.category_id
            LEFT JOIN reviews r ON r.product_id = p.id
            WHERE p.is_active = 1";
    $args = [];
    if (!empty($_GET['category'])) {
      $sql .= " AND c.name = ?";
      $args[] = $_GET['category'];
    }
    $sql .= " GROUP BY p.id, c.name ORDER BY c.id, p.name";
    $st = $pdo->prepare($sql);
    $st->execute($args);
    $rows = $st->fetchAll();
    foreach ($rows as &$r) {
      $r['price']       = (int) $r['price'];
      $r['final_price'] = final_price($r['price'], $user);
      $r['image']       = image_url($r['image']);
    }
    out([
      'categories' => $pdo->query("SELECT id, name FROM categories ORDER BY id")->fetchAll(),
      'products'   => $rows,
    ]);

  case 'POST':                                    // owner adds a product (multipart form)
    need_role(['owner']);
    $name  = trim($_POST['name'] ?? '');
    $cat   = (int) ($_POST['category_id'] ?? 0);
    $price = (int) ($_POST['price'] ?? 0);
    if ($name === '' || $cat < 1 || $price < 1) {
      out(['message' => 'name, category_id and price are required'], 400);
    }
    $file = null;
    if (!empty($_FILES['image']['name'])) {
      $f = $_FILES['image'];
      if ($f['error'] !== UPLOAD_ERR_OK || $f['size'] > 2 * 1024 * 1024) {
        out(['message' => 'Image upload failed or larger than 2 MB'], 400);
      }
      $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);
      $allowed = ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/webp' => 'webp'];
      $ext = $allowed[$mime] ?? null;
      if (!$ext) out(['message' => 'Only JPG, PNG or WebP images are allowed'], 400);
      $file = bin2hex(random_bytes(8)) . '.' . $ext;   // never trust the original name
      move_uploaded_file($f['tmp_name'], __DIR__ . '/../uploads/' . $file);
    }
    $st = $pdo->prepare(
      "INSERT INTO products (category_id, name, description, price, image)
       VALUES (?, ?, ?, ?, ?)");
    $st->execute([$cat, $name, trim($_POST['description'] ?? ''), $price, $file]);
    out(['message' => 'Product created', 'id' => (int) $pdo->lastInsertId()], 201);

  case 'PUT':                                     // owner changes a price
    need_role(['owner']);
    $id    = (int) ($_GET['id'] ?? 0);
    $price = (int) (body()['price'] ?? 0);
    if ($id < 1 || $price < 1) out(['message' => 'id and price are required'], 400);
    $st = $pdo->prepare("UPDATE products SET price = ? WHERE id = ?");
    $st->execute([$price, $id]);
```

```
    if ($st->rowCount() === 0) out(['message' => 'Product not found'], 404);
    out(['message' => 'Price updated']);

  default:
    out(['message' => 'Method not allowed'], 405);
}
```

Try it

Open `http://localhost/kopipiro/api/products.php` in the browser: you should see JSON with `price` equal to `final_price`.

Log in as the demo customer (Step 7), reload the same URL: `final_price` is now 10% lower.

Step 6 – Orders API

This is the core of the application. Read it slowly in class:

- Quantities are cleaned up, and prices are loaded from the database.
- The discount is applied only if the session user is a customer.
- The order goes to the cashier with the fewest unfinished orders.
- Order and items are saved in one database transaction: either both are saved or nothing is.
- GET filters by role: cashier sees `cashier_id` = me, customer sees `user_id` = me, owner sees all.

api/orders.php

```
<?php
require '../helpers.php';
$user = me();

switch ($_SERVER['REQUEST_METHOD']) {

  case 'POST':                                    // guest OR logged-in customer
    $in = body();
    $qty = [];                                    // product_id => quantity
    foreach (($in['items'] ?? []) as $it) {
      $pid = (int) ($it['product_id'] ?? 0);
      $q   = (int) ($it['qty'] ?? 0);
      if ($pid > 0 && $q > 0) $qty[$pid] = ($qty[$pid] ?? 0) + min($q, 50);
    }
    if (!$qty) out(['message' => 'Your order is empty'], 400);

    // Prices ALWAYS come from the database, never from the browser.
    $ph = implode(',', array_fill(0, count($qty), '?'));
    $st = $pdo->prepare(
      "SELECT id, name, price FROM products WHERE is_active = 1 AND id IN ($ph)");
    $st->execute(array_keys($qty));
    $prods = $st->fetchAll();
    if (count($prods) !== count($qty)) out(['message' => 'Some products are unavailable'], 400);

    $subtotal = 0;
    foreach ($prods as $p) $subtotal += $p['price'] * $qty[$p['id']];
    $discount = $subtotal - array_sum(array_map(
      fn($p) => final_price((int) $p['price'], $user) * $qty[$p['id']],
      $prods));
    $total = $subtotal - $discount;

    // Assign the cashier who currently has the fewest unfinished orders.
    $cashier = $pdo->query(
      "SELECT u.id FROM users u
       LEFT JOIN orders o ON o.cashier_id = u.id AND o.status <> 'done'
       WHERE u.role = 'cashier'
       GROUP BY u.id ORDER BY COUNT(o.id), u.id LIMIT 1")->fetchColumn();

    $code = 'KP-' . date('ymd') . '-' . strtoupper(bin2hex(random_bytes(3)));
```

```
    $guest = trim($in['customer_name'] ?? '') ?: 'Guest';
    $pdo->beginTransaction();
    try {
      $pdo->prepare(
        "INSERT INTO orders (order_code, user_id, customer_name, cashier_id,
                             subtotal, discount, total)
         VALUES (?, ?, ?, ?, ?, ?, ?)")
          ->execute([$code, $user['id'] ?? null, $user['name'] ?? $guest,
                     $cashier ?: null, $subtotal, $discount, $total]);
      $orderId = $pdo->lastInsertId();
      $ins = $pdo->prepare(
        "INSERT INTO order_items (order_id, product_id, product_name, unit_price, qty)
         VALUES (?, ?, ?, ?, ?)");
      foreach ($prods as $p) {
        $ins->execute([$orderId, $p['id'], $p['name'], $p['price'], $qty[$p['id']]]);
      }
      $pdo->commit();
    } catch (Throwable $e) {
      $pdo->rollBack();
      throw $e;
    }
    out(['order_code' => $code, 'subtotal' => $subtotal,
         'discount' => $discount, 'total' => $total], 201);

  case 'GET':                                       // each role sees a different slice
    $u = need_role(['owner', 'cashier', 'customer']);
    $sql = "SELECT o.id, o.order_code, o.customer_name, o.subtotal, o.discount, o.total,
                   o.status, o.created_at, c.name AS cashier,
                   (SELECT GROUP_CONCAT(CONCAT(i.qty, 'x ', i.product_name) SEPARATOR ', ')
                    FROM order_items i WHERE i.order_id = o.id) AS items
            FROM orders o LEFT JOIN users c ON c.id = o.cashier_id";
    $args = [];
    if ($u['role'] === 'cashier')  { $sql .= " WHERE o.cashier_id = ?"; $args[] = $u['id']; }
    if ($u['role'] === 'customer') { $sql .= " WHERE o.user_id = ?";    $args[] = $u['id']; }
    $st = $pdo->prepare($sql . " ORDER BY o.id DESC LIMIT 200");
    $st->execute($args);
    out($st->fetchAll());

  case 'PUT':                                       // update status
    $u = need_role(['owner', 'cashier']);
    $id = (int) ($_GET['id'] ?? 0);
    $status = body()['status'] ?? '';
    if (!in_array($status, ['pending', 'paid', 'done'], true)) {
      out(['message' => 'Invalid status'], 400);
    }
    $sql = "UPDATE orders SET status = ? WHERE id = ?";
    $args = [$status, $id];
    if ($u['role'] === 'cashier') { $sql .= " AND cashier_id = ?"; $args[] = $u['id']; }
    $st = $pdo->prepare($sql);
    $st->execute($args);
    if ($st->rowCount() === 0) out(['message' => 'Order not found'], 404);
    out(['message' => 'Status updated']);

  default:
    out(['message' => 'Method not allowed'], 405);
}
```

Step 7 – Storefront Page

One HTML page that consumes **our own API** with `fetch()`: product catalog with category filter, ratings, strike-through normal price for members, a cart, a checkout summary with discounted prices, and a login/register dialog. All text from the API is escaped before it is placed in the page (protection against XSS).

shop.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```
<title>Kopipiro</title>
<style>
  body { font-family: system-ui, sans-serif; margin: 0; background: #faf6f1; color: #3b2a1e; }
  header { display: flex; justify-content: space-between; align-items: center;
           padding: 12px 24px; background: #3b2a1e; color: #fff; }
  header a { color: #f3e3c7; margin-right: 12px; }
  #promo { margin: 0; padding: 8px 24px; background: #f3e3c7; }
  main { display: grid; grid-template-columns: 1fr 320px; gap: 24px; padding: 24px; }
  #grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
          gap: 16px; margin-top: 12px; }
  .card { background: #fff; border-radius: 10px; padding: 12px; box-shadow: 0 1px 4px #0002; }
  .card img { width: 100%; height: 130px; object-fit: cover; border-radius: 6px; background: #eee;
}
  .old { text-decoration: line-through; color: #999; margin-right: 6px; }
  aside { background: #fff; border-radius: 10px; padding: 16px; height: fit-content; }
  #cats button.on { font-weight: bold; }
  button { cursor: pointer; }
</style>
</head>
<body>
<header>
  <h1>Kopipiro</h1>
  <div>
    <a href="dashboard.html">Staff dashboard</a>
    <span id="who"></span> <button id="btnAuth"></button>
  </div>
</header>
<p id="promo"></p>

<main>
  <section>
    <div id="cats"></div>
    <div id="grid"></div>
  </section>
  <aside>
    <h2>Checkout</h2>
    <ul id="cart"></ul>
    <p>Total: <b id="total">Rp 0</b></p>
    <input id="guestName" placeholder="Your name (guests)">
    <button id="btnCheckout">Place order</button>
    <div id="result"></div>
  </aside>
</main>

<dialog id="dlg">
  <h3>Log in / Register</h3>
  <p><input id="aName" placeholder="Name (register only)"></p>
  <p><input id="aEmail" type="email" placeholder="Email"></p>
  <p><input id="aPass" type="password" placeholder="Password"></p>
  <button id="btnLogin">Log in</button>
  <button id="btnReg">Register</button>
  <button id="btnCancel">Cancel</button>
  <p id="aMsg"></p>
</dialog>

<script>
const $ = s => document.querySelector(s);
const esc = s => String(s ?? '').replace(/[&<>"']/g,
  c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
const rp = n => 'Rp ' + Number(n).toLocaleString('id-ID');

let me = { role: 'guest' }, category = '', cart = {}, known = {};

async function api(url, opts = {}) {
  const res = await fetch(url, { headers: { 'Content-Type': 'application/json' }, ...opts });
  const data = await res.json();
  if (!res.ok) throw new Error(data.message || 'Request failed');
  return data;
}

async function init() {
  me = await api('api/auth.php?action=me');
  $('#who').textContent = me.name ? me.name + ' (' + me.role + ')' : 'Guest';
```

```
  $('#btnAuth').textContent = me.name ? 'Log out' : 'Log in';
  $('#promo').textContent = me.role === 'customer'
    ? 'Member discount active: 10% off every item!'
    : 'Register and log in as a customer to get 10% off.';
  await loadProducts();
}

async function loadProducts() {
  const q = category ? '?category=' + encodeURIComponent(category) : '';
  const data = await api('api/products.php' + q);
  data.products.forEach(p => known[p.id] = p);          // remember prices for the cart
  $('#cats').innerHTML = ['', ...data.categories.map(c => c.name)].map(c =>
    `<button class="${c === category ? 'on' : ''}" data-c="${esc(c)}">${esc(c || 'All')}</button>`
  ).join(' ');
  $('#grid').innerHTML = data.products.map(p => `
    <div class="card">
      <img src="${esc(p.image || '')}" alt="">
      <h3>${esc(p.name)}</h3>
      <small>${esc(p.category)} | rating ${p.rating ?? '-'} (${p.reviews})</small>
      <p>${esc((p.description || '').slice(0, 100))}</p>
      <p>${p.final_price < p.price ? `<span class="old">${rp(p.price)}</span>` : ''}
         <b>${rp(p.final_price)}</b></p>
      <button data-add="${p.id}">Add</button>
    </div>`).join('');
  renderCart();
}

function renderCart() {
  let total = 0;
  $('#cart').innerHTML = Object.entries(cart).map(([id, qty]) => {
    const p = known[id];
    const line = p.final_price * qty;
    total += line;
    return `<li>${qty} x ${esc(p.name)} @ ${rp(p.final_price)} = ${rp(line)}
            <button data-rm="${id}">x</button></li>`;
  }).join('');
  $('#total').textContent = rp(total);
}

document.addEventListener('click', async e => {
  const d = e.target.dataset;
  if (d.c !== undefined) { category = d.c; loadProducts(); }
  if (d.add) { cart[d.add] = (cart[d.add] || 0) + 1; renderCart(); }
  if (d.rm) { delete cart[d.rm]; renderCart(); }
  if (e.target.id === 'btnPay') {
    try {
      const r = await api('api/pay.php', { method: 'POST',
        body: JSON.stringify({ order_code: d.code }) });
      location.href = r.redirect_url;                   // Midtrans hosted payment page
    } catch (err) { $('#result').textContent = err.message; }
  }
});

$('#btnCheckout').onclick = async () => {
  const items = Object.entries(cart).map(([id, qty]) => ({ product_id: +id, qty }));
  try {
    const o = await api('api/orders.php', { method: 'POST',
      body: JSON.stringify({ items, customer_name: $('#guestName').value }) });
    cart = {}; renderCart();
    $('#result').innerHTML = `Order <b>${esc(o.order_code)}</b><br>
      Subtotal ${rp(o.subtotal)}<br>Discount ${rp(o.discount)}<br>
      Total <b>${rp(o.total)}</b><br>
      <button id="btnPay" data-code="${esc(o.order_code)}">Pay online (sandbox)</button>`;
  } catch (err) { $('#result').textContent = err.message; }
};

async function auth(action) {
  try {
    const r = await api('api/auth.php?action=' + action, { method: 'POST',
      body: JSON.stringify({ name: $('#aName').value, email: $('#aEmail').value,
                             password: $('#aPass').value }) });
    if (action === 'login') { $('#dlg').close(); cart = {}; init(); }
    else $('#aMsg').textContent = r.message;
```

```
  } catch (err) { $('#aMsg').textContent = err.message; }
}
$('#btnLogin').onclick = () => auth('login');
$('#btnReg').onclick = () => auth('register');
$('#btnCancel').onclick = () => $('#dlg').close();
$('#btnAuth').onclick = async () => {
  if (me.name) {
    await api('api/auth.php?action=logout', { method: 'POST' });
    cart = {}; known = {}; init();
  } else $('#dlg').showModal();
};

init();
</script>
</body>
</html>
```

Try it

Add products as a guest and place an order: discount is Rp 0.

Log in as the demo customer, place the same order: 10% discount.

Register a new account in the dialog, then log in with it.

Step 8 – Dashboard, Charts, and Excel Export

Cashiers and the owner log in on the dashboard. A cashier sees only the orders assigned to them; the owner sees all orders, two charts (daily revenue and top products), an Excel download, and a form to add products with an image.

api/sales.php

```
<?php
require '../helpers.php';
need_role(['owner']);

out([
  'daily' => $pdo->query(
    "SELECT DATE(created_at) AS day, COUNT(*) AS orders, SUM(total) AS revenue
     FROM orders WHERE status IN ('paid','done')
     GROUP BY DATE(created_at) ORDER BY day")->fetchAll(),
  'top' => $pdo->query(
    "SELECT i.product_name, SUM(i.qty) AS qty
     FROM order_items i JOIN orders o ON o.id = i.order_id
     WHERE o.status IN ('paid','done')
     GROUP BY i.product_name ORDER BY qty DESC LIMIT 5")->fetchAll(),
]);
```

dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Kopipiro Dashboard</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<style>
  body { font-family: system-ui, sans-serif; margin: 24px; }
  table { border-collapse: collapse; width: 100%; }
  td, th { border: 1px solid #ccc; padding: 6px; text-align: left; }
  .charts { display: flex; gap: 24px; }
  .charts div { width: 45%; }
  .hidden { display: none; }
</style>
</head>
<body>
<h1>Kopipiro Dashboard</h1>
<p><span id="who"></span> <a href="shop.html">Back to shop</a></p>

<form id="loginForm" class="hidden">
```

```
  <input id="email" type="email" placeholder="Email">
  <input id="pass" type="password" placeholder="Password">
  <button>Log in</button> <span id="loginMsg"></span>
</form>

<section id="ordersBox" class="hidden">
  <h2 id="ordersTitle"></h2>
  <table>
    <thead><tr><th>Code</th><th>Date</th><th>Customer</th><th>Items</th>
      <th>Total</th><th>Cashier</th><th>Status</th></tr></thead>
    <tbody id="rows"></tbody>
  </table>
</section>

<section id="ownerBox" class="hidden">
  <h2>Sales</h2>
  <p><a href="api/export.php">Download all orders (Excel)</a></p>
  <div class="charts">
    <div><canvas id="c1"></canvas></div>
    <div><canvas id="c2"></canvas></div>
  </div>

  <h2>Add product</h2>
  <form id="prodForm">
    <input name="name" placeholder="Product name" required>
    <select name="category_id" id="catSel"></select>
    <input name="price" type="number" min="1" placeholder="Price (Rp)" required>
    <input name="description" placeholder="Description">
    <input name="image" type="file" accept="image/jpeg,image/png,image/webp">
    <button>Add</button> <span id="pMsg"></span>
  </form>
</section>

<script>
const $ = s => document.querySelector(s);
const esc = s => String(s ?? '').replace(/[&<>"']/g,
  c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
const rp = n => 'Rp ' + Number(n).toLocaleString('id-ID');
let me = {};

async function api(url, opts = {}) {
  const isForm = opts.body instanceof FormData;       // let the browser set multipart headers
  const headers = isForm ? {} : { 'Content-Type': 'application/json' };
  const res = await fetch(url, { headers, ...opts });
  const data = await res.json();
  if (!res.ok) throw new Error(data.message || 'Request failed');
  return data;
}

async function init() {
  me = await api('api/auth.php?action=me');
  const staff = ['owner', 'cashier'].includes(me.role);
  $('#who').textContent = me.name ? me.name + ' (' + me.role + ')' : 'Not logged in';
  $('#loginForm').classList.toggle('hidden', staff);
  $('#ordersBox').classList.toggle('hidden', !staff);
  $('#ownerBox').classList.toggle('hidden', me.role !== 'owner');
  if (!staff) return;
  loadOrders();
  if (me.role === 'owner') { loadCharts(); loadCategories(); }
}

async function loadOrders() {
  const rows = await api('api/orders.php');
  $('#ordersTitle').textContent = me.role === 'owner' ? 'All orders' : 'My assigned orders';
  $('#rows').innerHTML = rows.map(o => `<tr>
    <td>${esc(o.order_code)}</td><td>${esc(o.created_at)}</td><td>${esc(o.customer_name)}</td>
    <td>${esc(o.items)}</td><td>${rp(o.total)}</td><td>${esc(o.cashier ?? '-')}</td>
    <td><select data-id="${o.id}">${['pending', 'paid', 'done'].map(s =>
      `<option ${s === o.status ? 'selected' : ''}>${s}</option>`).join('')}</select></td>
  </tr>`).join('');
}
```

```
$('#rows').addEventListener('change', async e => {
  await api('api/orders.php?id=' + e.target.dataset.id,
    { method: 'PUT', body: JSON.stringify({ status: e.target.value }) });
});

async function loadCharts() {
  const s = await api('api/sales.php');
  new Chart($('#c1'), { type: 'bar', data: { labels: s.daily.map(r => r.day),
    datasets: [{ label: 'Revenue (Rp)', data: s.daily.map(r => +r.revenue) }] } });
  new Chart($('#c2'), { type: 'pie', data: { labels: s.top.map(r => r.product_name),
    datasets: [{ data: s.top.map(r => +r.qty) }] } });
}

async function loadCategories() {
  const d = await api('api/products.php');
  $('#catSel').innerHTML = d.categories.map(c =>
    `<option value="${c.id}">${esc(c.name)}</option>`).join('');
}

$('#prodForm').onsubmit = async e => {
  e.preventDefault();
  try {
    await api('api/products.php', { method: 'POST', body: new FormData(e.target) });
    $('#pMsg').textContent = 'Product added';
    e.target.reset();
  } catch (err) { $('#pMsg').textContent = err.message; }
};

$('#loginForm').onsubmit = async e => {
  e.preventDefault();
  try {
    await api('api/auth.php?action=login', { method: 'POST',
      body: JSON.stringify({ email: $('#email').value, password: $('#pass').value }) });
    init();
  } catch (err) { $('#loginMsg').textContent = err.message; }
};

init();
</script>
</body>
</html>
```

The Excel export uses the PhpSpreadsheet library. Install it with Composer (install Composer from getcomposer.org first), and make sure `extension=zip` and `extension=gd` are enabled in `php.ini`:

```
cd C:\xampp\htdocs\kopipiro
composer require phpoffice/phpspreadsheet
```

api/export.php

```
<?php
require '../helpers.php';
require '../vendor/autoload.php';                // composer require phpoffice/phpspreadsheet

use PhpOffice\PhpSpreadsheet\Spreadsheet;
use PhpOffice\PhpSpreadsheet\Writer\Xlsx;

need_role(['owner']);

$rows = $pdo->query(
  "SELECT o.order_code, o.created_at, o.customer_name, c.name AS cashier,
          o.subtotal, o.discount, o.total, o.status
   FROM orders o LEFT JOIN users c ON c.id = o.cashier_id
   ORDER BY o.id DESC")->fetchAll();

$book  = new Spreadsheet();
$sheet = $book->getActiveSheet();
$sheet->setTitle('Orders');
$sheet->fromArray(['Order code', 'Date', 'Customer', 'Cashier',
```

```
                  'Subtotal', 'Discount', 'Total', 'Status'], null, 'A1');
$sheet->fromArray(array_map('array_values', $rows), null, 'A2');

header('Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet');
header('Content-Disposition: attachment; filename="kopipiro-orders.xlsx"');
(new Xlsx($book))->save('php://output');
```

> **Remember**
> Charts only count orders with status **paid** or **done**. As cashier or owner, change a few orders to `paid` in the dashboard before checking the charts.

Step 9 – Online Payment with the Midtrans Sandbox

Payment gateways are APIs too. The flow has three parts, and it needs no real money or business documents in sandbox mode:

1. Our server calls the gateway API to create a payment and receives a payment page URL (`api/pay.php`).
2. The customer pays on the gateway's hosted page (in sandbox, with test payment simulators).
3. The gateway calls **our** API (a webhook) to tell us the result (`api/midtrans_notify.php`). We verify the signature before updating the order.

Setup

- Create a free Midtrans account and use the **Sandbox** environment.
- Go to Settings → Access Keys and copy the sandbox **Server Key** into `MIDTRANS_SERVER_KEY` in `config.php`. Never put the server key in HTML or JavaScript, and never publish it.
- Pay using the test payment methods described in the Midtrans sandbox testing documentation.
- For the webhook: Midtrans must reach your computer from the internet, so run a tunnel such as ngrok (`ngrok http 80`) and set the Payment Notification URL in the Midtrans dashboard to `https://YOUR-TUNNEL/kopipiro/api/midtrans_notify.php`.

api/pay.php

```
<?php
require '../helpers.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') out(['message' => 'Method not allowed'], 405);

$st = $pdo->prepare("SELECT * FROM orders WHERE order_code = ? AND status = 'pending'");
$st->execute([body()['order_code'] ?? '']);
$order = $st->fetch();
if (!$order) out(['message' => 'Pending order not found'], 404);

$payload = [
  'transaction_details' => ['order_id' => $order['order_code'],
                            'gross_amount' => (int) $order['total']],
  'customer_details'    => ['first_name' => $order['customer_name']],
];

$ch = curl_init('https://app.sandbox.midtrans.com/snap/v1/transactions');
curl_setopt_array($ch, [
  CURLOPT_POST           => true,
  CURLOPT_RETURNTRANSFER => true,
  CURLOPT_TIMEOUT        => 15,
  CURLOPT_USERPWD        => MIDTRANS_SERVER_KEY . ':',   // Basic auth: key as username
  CURLOPT_HTTPHEADER     => ['Content-Type: application/json', 'Accept: application/json'],
  CURLOPT_POSTFIELDS     => json_encode($payload),
]);
$resp = curl_exec($ch);
$code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
```

```
curl_close($ch);

$data = json_decode((string) $resp, true);
if ($code !== 201 || empty($data['redirect_url'])) {
  out(['message' => 'Payment gateway error'], 502);
}
out(['redirect_url' => $data['redirect_url']]);
```

api/midtrans_notify.php

```
<?php
require '../helpers.php';
$n = body();                                        // Midtrans POSTs JSON here

// 1. Is this really from Midtrans? SHA512(order_id + status_code + gross_amount + server key)
$expected = hash('sha512',
  ($n['order_id'] ?? '') . ($n['status_code'] ?? '') .
  ($n['gross_amount'] ?? '') . MIDTRANS_SERVER_KEY);
if (!hash_equals($expected, $n['signature_key'] ?? '')) {
  out(['message' => 'Invalid signature'], 403);
}

// 2. Success = code 200 + (settlement, or capture with fraud status accept)
$status = $n['transaction_status'] ?? '';
$fraud  = $n['fraud_status'] ?? 'accept';
$paid = ($n['status_code'] ?? '') === '200'
     && ($status === 'settlement' || ($status === 'capture' && $fraud === 'accept'));

if ($paid) {
  // Also check the amount, and only touch pending orders (safe if notified twice).
  $pdo->prepare("UPDATE orders SET status = 'paid'
                 WHERE order_code = ? AND total = ? AND status = 'pending'")
      ->execute([$n['order_id'], (int) $n['gross_amount']]);
}
out(['ok' => true]);
```

No tunnel? No problem

Without a public URL the webhook cannot arrive. The cashier can then set the order to `paid` on the dashboard, which is also how cash payments at the counter work. Midtrans also offers a status-check API as an alternative to the webhook.

Xendit and DOKU also provide free sandboxes. The pattern is identical; only `pay.php` and the notification check change.

6.4 Testing Checklist

Scenario	Expected result
Guest opens the shop	Normal prices, promo message asks to log in
Demo customer logs in	Prices 10% lower, normal price struck through
Guest places an order	Discount = 0, order has no user
Customer places an order	Discount = 10% of subtotal
Send an order with a fake `price` field using Postman	Ignored: the server uses database prices
Cashier Ani logs in	Sees only orders assigned to Ani
Cashier Ani opens `api/sales.php`	403 Forbidden
Customer sends POST to `api/products.php`	403 Forbidden

Scenario	Expected result
Owner adds a product with an image	Appears in the shop with the uploaded picture
Owner uploads a `.php` file as the image	400, rejected by MIME type check
Owner marks orders as paid	Charts update; Excel download opens correctly

6.5 Security Notes

- Prepared statements everywhere (SQL injection).
- `password_hash` / `password_verify` for passwords; never store plain passwords.
- Prices and discounts calculated on the server only.
- Role checks inside every protected endpoint, not only in the interface.
- Upload checks: size, real MIME type, random file name, allowed extensions only.
- Output escaped in the browser (`esc()`), so product text cannot inject scripts.
- Webhook signature verified with `hash_equals`, and the amount is checked.
- Secrets (server key) stay on the server.

Not covered, and worth discussing as next steps: CSRF protection, login rate limiting, HTTPS and secure cookie flags, and moving the server key into an environment file.

6.6 Extension Exercises

1. Add `POST api/reviews.php` so a logged-in customer can rate a product (1 to 5) and leave a comment.
2. Add a customer page that lists the customer's own order history (the API already supports it).
3. Let the owner edit prices from the dashboard using `PUT api/products.php?id=`.
4. Use the Indonesian regions API (`emsifa.github.io/api-wilayah-indonesia`) to add a delivery address dropdown at checkout (province, city, district).
5. Show an "iced drink suggestion" banner when the Open-Meteo API reports a temperature above 30 °C.
6. Replace Midtrans with Xendit or DOKU sandbox and compare the two APIs.

6.7 Assessment Rubric for the Project

Aspect	Weight	Criteria
Roles and permissions	25%	Owner, cashier and customer see and do only what they should
Discount and checkout logic	20%	Correct for guests and customers; server-side calculation
Product management	15%	Add product, image upload, price change
Reports	15%	Working charts and Excel download
Public API / payment integration	15%	Seed from public API; sandbox payment flow

Aspect	Weight	Criteria
Security and code quality	10%	Prepared statements, escaping, clear structure

Appendix A – Quiz Answer Key

1. API = Application Programming Interface; it acts as an intermediary so one application can use another's data or functions without knowing its internals.
2. GET (Read), POST (Create), PUT (Update), DELETE (Delete).
3. 201 = data was created; 400 = the client's request was wrong or incomplete; 404 = data or endpoint not found.
4. REST usually uses JSON (XML is possible) and is more flexible; SOAP requires XML with a strict WSDL contract.
5. `json_encode()` (array → JSON) and `json_decode()` (JSON → array/object).
6. Because third parties can fail, time out, change format, or return malicious content; check errors/status codes and escape output.

Appendix B – XAMPP Troubleshooting

Problem	Solution
`Call to undefined function curl_init()`	Enable `extension=curl` in php.ini, then restart Apache.
cURL error 60: SSL certificate problem	Download `cacert.pem` from curl.se/docs/caextract.html, save it in `C:\xampp\php\extras\ssl\`, then set `curl.cainfo` and `openssl.cafile` in php.ini to that file. Restart Apache. Avoid disabling SSL verification (`CURLOPT_SSL_VERIFYPEER=false`) except briefly for local tests.
`file_get_contents` fails for https	Make sure `extension=openssl` is enabled in php.ini.
Port 80 conflict (Apache won't start)	Stop the program using port 80 (Skype, IIS) or change the port in httpd.conf.
PUT/DELETE does not receive the body	Read the body with `file_get_contents('php://input')`, not `$_POST`.
JSON output looks messy in the browser	Make sure `header('Content-Type: application/json')` is set and nothing is echoed before `<?php`.

Appendix C – Public APIs for Practice

API	Base URL	Notes
JSONPlaceholder	https://jsonplaceholder.typicode.com	Fake data (posts, users, todos) for learning
Open-Meteo	https://api.open-meteo.com	Weather, no API key
REST Countries	https://restcountries.com	Country data
Open Library	https://openlibrary.org/developers/api	Book data

API	Base URL	Notes
Sample APIs Coffee	https://api.sampleapis.com/coffee/hot	Coffee names, descriptions, ingredients (used in Section 6)
Indonesian regions	https://emsifa.github.io/api-wilayah-indonesia/api/provinces.json	Provinces, cities, districts, villages
Midtrans Sandbox	https://docs.midtrans.com	Free test payments (no real money)
Xendit / DOKU Sandbox	https://docs.xendit.co | https://developers.doku.com	Alternative sandbox payment gateways